

5-Day Python Learning Track

Audience: This is for new interns coming from Java or C++.

Learning Expectation: Not Python syntax. The judgment that makes code readable, deployable, and maintainable inside Frappe and ERPNext. Even the code you write with an LLM.

Introduction

The syntax is the cheapest part of this job now. An LLM will write the loop, the class, the SQL query for you. What it will not reliably do is decide where a piece of logic belongs, what a good abstraction looks like, when a class is actually worth it, or how to stop a module from turning into a swamp. That part is on you.

So this week is not about memorising Python. You already think in types, objects, and structure. That carries over. What you need to drop are the Java and C++ habits that produce ugly Python, and what you need to build is **taste**. The ability to look at generated code and say "**this is production-ready**" or "**this needs to be reshaped before it goes anywhere.**"

Here is the deal. By the end of the week, you should be able to take a messy 40-line blob, yours or an LLM's, and turn it into small, typed, testable pieces organised into a clean package. And you should be able to explain why you made each choice. If you can do that, you are ready for the Frappe week.

Every day is a full day. If something pulls you deeper, go deeper. **Nobody is graded on finishing fast.** The 5 days are a floor, not a ceiling.

Day 1, Rewire the mental model

Goal: drop the Java and C++ reflexes that make Python ugly, and set up a clean environment.

Block A, Python's object model. Everything is an object; there are no primitives. Names are references, not boxes. Mutable vs immutable (list and dict vs tuple, str, int). `is` vs `==`. Dynamic typing and duck typing, if it walks like a duck. This is where most of your early bugs will come from, so sit with it. Mutable references and shared state behave differently from what you are used to in Java and C++.

Block B, Pythonic idioms Truthiness. Comprehensions instead of manual loops. Tuple unpacking, `enumerate`, `zip`. Ask forgiveness (EAFP) vs look before you leap (LBYL). Context managers (`with`). f-strings. Run `import this` and read the Zen of Python, there is one obvious way to do things. Idiomatic Python is shorter and clearer, and it is what LLMs produce. Learn to recognize it, do not fight it.

Block C, Environment done right `python -m venv`, activation, `pip`, `requirements.txt` and `pip freeze`. Why global installs are a trap. A quick look at `pyproject.toml`. Editable installs with `pip install -e ..`

Frappe tie-in: A Frappe app is a Python package sitting inside bench's virtualenv. Bench manages the env for you, but you need to understand what it is managing.

Exercise: Spin up a venv, install a package, write a small script using a comprehension and a context manager. Then take a Java-style snippet, index loops and getters everywhere, and rewrite it the Pythonic way.

Day 2, Functions and abstractions

Goal: write small, honest functions with a clear contract.

Block A, Function design One function, one job. Pure functions vs ones with side effects, and why you keep the side effects at the edges. Arguments: positional, keyword, defaults, `*args`, `**kwargs`, keyword-only. Return early, do not nest five levels deep. And learn the mutable default argument trap, `def f(x=[])`, because it will bite you.

Block B, Type hints and contracts Annotations, the `typing` toolkit (`Optional`, `list[...]`, `dict[...]`, `Union`, `Callable`). Here is why hints matter so much now: they make generated code reviewable at a glance, and they let tooling catch the mistakes the model made. Docstrings that say what and why, not how.

Block C, Abstraction and DRY Naming is design. Extract functions. The rule of three, do not abstract until you have seen it three times. And watch the opposite mistake, abstracting too early. First-class functions, closures, and decorators (just an intro, because Frappe leans on these heavily).

Frappe tie-in: `@frappe.whitelist()` is a decorator. Hooks are functions. Controller methods are just functions on a class. Everything today shows up immediately.

Exercise: Take a deliberately messy 40-line function and break it into a handful of small, typed, documented functions.

Day 3, OOP the Python way

Goal: use classes when they are actually worth it. Do not Java-ify everything.

Block A, Classes in Python `__init__`, `self`, instance vs class attributes. There is no `private` or `protected` keyword, the convention is a leading underscore. Use properties instead of getter and setter boilerplate. That one habit alone deletes half the code a Java dev writes on reflex.

Block B, Composition over inheritance When inheritance is right and when it is a trap. MRO basics, mixins. `@dataclass` for data holders. `Enum`, `NamedTuple`. Dunder methods (`__repr__`, `__eq__`, `__len__`, `__str__`), and why a good `__repr__` saves you hours of debugging.

Block C, When not to use a class Modules work fine as namespaces. A function beats a single-method class. SOLID, adapted lightly to Python. Your instinct will be to wrap everything in a class. Fight it.

Frappe tie-in: A DocType controller is a class that subclasses `Document`. Lifecycle hooks (`validate`, `before_save`, `on_submit`) are methods. Understand where composition beats inheritance when you are customizing behavior.

Exercise: Model a simple ERPNext-style entity, say an Invoice, two ways. A `@dataclass` for the data, and a controller-style class with a `validate()` method. Then tell me which belongs where and why.

Day 4, Modules, packages, and project structure

Goal: organize code so it is easy to navigate, import, and deploy.

Block A, Modules and imports Module is a file, package is a folder with `__init__.py`.

Absolute vs relative imports. How import resolution actually works. Avoiding circular imports, and you will hit this in Frappe. `if __name__ == "__main__"`.

Block B, Layout and separation of concerns A sensible project layout. Layering: API, business logic, and data access as separate layers. Keep I/O at the edges and logic in the middle. Config separation, use env vars, never hardcode secrets. Dependency management and pinning.

Block C, Reusability and packaging Build a small installable package. Entry points. Basic versioning discipline.

Frappe tie-in: The anatomy of a Frappe app, `hooks.py`, module folders, `doctype/` controllers, `api.py`. Where business logic should live vs what leaks into controllers. This is the single biggest quality lever in a real Frappe codebase, so pay attention here.

Exercise: Take a single-file script and split it into a proper package with clean layers. Then import it and use it from a separate script.

Day 5, Production hardening and working with LLMs

Goal: write code that survives contact with production, and with AI assistance.

Block A, Errors, logging, defensiveness Exceptions vs error codes. Custom exception classes. `try`, `except`, `finally`, and never swallow an error silently. Logging instead of `print`. Validate input at the boundaries.

Block B, Testing and quality tooling Unit tests (`pytest` or `unittest`) and how to structure a test. `ruff`, a formatter, `mypy`, pre-commit hooks, reading a diff. The point that matters most: tests are what make it safe to let an LLM refactor your code.

Block C, Working with LLMs the right way Treat generated code as a draft, not an answer. Review it against everything from Days 1 to 4. Right abstraction? Right layer? Typed? Errors handled? Prompt with context and constraints. Catch hallucinated APIs. Refactor the output into the house style. Ship small, verifiable commits.

Frappe tie-in: Frappe's test framework and `ruff` config. How to review AI-generated Frappe code specifically, whitelist safety, permission checks, SQL injection risk, N+1 query patterns.

Exercise: Take an LLM-generated snippet, list everything wrong with it (types, error handling, structure, Frappe-specific risks), refactor it, and add one test.

Checklist "did you get it?"

- [] Can rewrite a Java-style loop or getter block into idiomatic Python and explain why
- [] Writes small, typed, single-responsibility functions by default
- [] Knows when not to reach for a class
- [] Can split a script into a layered package with clean imports
- [] Handles errors and logs on purpose (no bare `except`, no `print`)
- [] Can look at LLM output and name what is wrong with it before running it